

3회차 복습 카드 — 집에서 오늘 순서 그대로

오늘 놓친 부분이 있어도 괜찮아요. 이 카드만 있으면 혼자 처음부터 끝까지 다시 할 수 있습니다.

회색 상자 안의 문장을 그대로 복사해서 Claude에게 붙여넣고 Enter만 누르면 돼요.

💡 실행은 오늘과 똑같이 **Claude Desktop 앱 Code** 탭에서. 지난주 만든 미니 당근 폴더를 열고 시작하세요.

🎯 **오늘의 목적지:** 미니 당근에 실시간 채팅 + 보안을 붙이기. 그런데 진짜 볼 건 채팅이 아니라 — **지난주 그 공정(구상→설계→발사→검수)을 도구가 대신 밟는 것.**

시작 전에 — 오늘의 큰 그림

지난주엔 공정을 손으로 밟았죠(PRD 쓰기·checklist·직접 검수). 오늘은 그걸 **도구가** 대신합니다. 크게 **둘만** 배워요:

1. **brainstorming** — AI가 캐물어서 설계도·계획서를 만들어줌
2. **TDD** — 만들기 전에 "합격 조건"부터 정하고 만듦

💡 도구는 새것, **순서(공정)**는 지난주랑 똑같아요. 그래서 지난주에 손으로 해본 게 힘이 됩니다.

STEP 1. superpowers — 점검하고 설치하기

superpowers는 프로들이 미리 만들어둔 **스킬 상자**(여러 개가 한 번에)예요. 그런데 좋아 보인다고 바로 설치하지 않습니다.

⚠️ **오늘의 보안 습관:** 인터넷에서 받은 걸 그냥 실행 = 모르는 사람이 준 USB를 회사 컴퓨터에 꽂는 것. **뭘 설치하느냐가 이미 보안이에요.**

① 먼저 깃허브에서 **4점검** (github.com/obra/superpowers 열어서):

- 누가 만들었나 · 몇 명 쓰나(별) · 살아있나(최근 갱신) · 라이선스

② AI에게 점검부터 시키기:

`https://github.com/obra/superpowers` 이거 설치하려고 해.
설치하기 전에 먼저: 어떤 기능들이 있는지 요약해주고,
보안상 위험한 부분은 없는지(외부로 뭘 보내거나 이상한 권한을 요구하거나) 점검해줘.
아직 설치하지는 말고 보고만 해줘

✅ **확인:** 기능 요약 + 위험 여부 보고가 나오면 성공. **아직 설치 안 됐습니다.**

③ 점검 결과가 괜찮으면 그때 설치:

점검해보니 괜찮네. 이 **superpowers** 플러그인 설치해줘.
설치 방법 찾아서 진행해줘 (github.com/obra/superpowers)

✅ **확인:** 슬래시(/) 목록에 새 명령들이 늘어나 있으면 성공.

💡 혹시 설치가 막히면 → AI에게 *"플러그인 마켓플레이스로 설치하는 법 찾아줘 (obra/superpowers-marketplace)"* 라고 부탁드립니다.

STEP 2. brainstorming — 설계부터 AI와 같이

지난주엔 제가 PRD를 써서 드렸죠. 오늘은 AI가 캐물어서 같이 설계합니다.

미니당근에 두 가지를 추가하고 싶어. brainstorming으로 같이 구상하자.

1. 상품마다 판매자와 1:1로 대화하는 실시간 채팅
2. 상품은 올린 사람만 수정·삭제할 수 있게 하는 보안 규칙

✅ **확인:** AI가 질문을 하나씩 던지기 시작하면 성공. 열 번 넘게 답하게 되는 게 정상이에요. 길다고 당황 마세요.

- 객관식이 많으니 번호로 답하면 돼요. 모르겠으면 AI가 추천한 걸 고르세요.
- 중간에 *"이렇게 할까요?"* 물어보면 괜찮으면 좋다고, 아니면 고쳐달라고.
- 문서를 git에 커밋하겠다고 할 수도 있어요 — 놀라지 말고 허락하세요.

💡 왜 이렇게 캐물죠? "채팅 만들어줘"라고 그냥 시켰으면, AI가 "판매자가 누구인지"를 어디선가 지어냈을 거예요. 질문이 진짜 문제(로그인이 없는데 '나'를 어떻게 구분?)를 먼저 꺼내줍니다.

✅ **확인:** 다 답하고 나면 계획서(할 일이 체크박스로 정리된 문서)가 나오면 성공.

STEP 3. 발사 — 계획대로, 테스트 먼저(TDD)

계획서가 나오면 AI가 "어떻게 진행할까요?"라고 물어요. 그때 이렇게 답하는 게 발사예요.

바로 진행해줘. TDD로 진행해줘.

한 단계 끝날 때마다 커밋하고, 그 단계 이름으로 태그도 달아줘.

모든 구현이 끝나면 단계별로 뭘 했는지 한 줄씩 보고하고, 로컬 서버로 띄워서 브라우저로 열어줘

✅ **확인:** 구현 로그가 흐르기 시작하면 정상. 끝나길 기다리지 말고, 도는 동안 아래 [TDD가 뭐죠?]를 읽으세요.

🧩 [TDD가 뭐죠?] — 오늘 제일 핫갈린 것, 천천히

한 문장은 이거예요: "만들기 전에, 이렇게 되면 합격"을 먼저 정한다." (요리 레시피의 "이 맛이 나면 완성"처럼.) 순서가 낫설 뿐이에요. 미니당근 채팅으로 따라가 봅시다.

등장인물이 둘입니다 — 핫갈리면 안 돼요.

- 📋 **채점표** = "결과가 맞는가?"를 확인해주는 코드 (테스트)
- ⚙️ **기능** = 그 결과를 실제로 만들어내는 코드

① 채점표를 먼저 적는다 (기능은 아직 안 만듦)

"안녕하세요"를 보낸다 → 목록에 "안녕하세요"가 **떠야** 한다

보내는 기능은 아직 한 줄도 안 만들었어요. 합격 조건부터 못 박는 겁니다.

② 실행하면 → 빨강 (불합격) — 그게 정상

당연해요. 보내는 기능이 없으니 목록에 안 뜹니다.

-  **틀린 코드를 일부러 짤 게 아니에요.** 기능 자리가 아직 빈칸일 뿐.
- 이 빨강이 왜 중요하냐면 → 채점표가 진짜 "안녕하세요"를 **확인하고** 있다는 증거거든요. 빨강을 한 번도 못 보면, 채점표가 엉뚱한 걸 보고 있거나 오타로 아무것도 안 보고 있어도 눈치 못 챍니다.

③ 이제 기능을 최소로 만든다 → 실행 → 초록 (합격)

보낸 메시지를 목록에 추가 → 다시 실행 → 목록에 "안녕하세요"가 뜸. **합격!**

왜 굳이 먼저 하죠?

기능부터 다 만들고 채점표를 나중에 쓰면, 이미 뜨니까 **바로 초록**이에요. 그럼 그 채점표가 진짜 일하는지 **한 번도** 못 본 거죠. (자물쇠를 채워본 적 없는데 "안 열리네, 안전하네" 하는 셈)

빨강 → 초록이 한 번에 되나요?

아니에요. 빨강 → 고침 → 여전히 빨강 → 고침 → 초록. **이 왕복이 정상**입니다.

그래서 얻는 것:

나중에 빨간불이 뜨면 "코드가 틀렸나, 테스트가 틀렸나?" 하고 안 헤매요. 채점표는 이미 검증됐으니 → 코드 탓인 걸 바로 압니다.

 지난주 `/check` 가 "만들고 나서 검사"였다면, TDD는 "만들기 전에 기준부터"예요. 한 발 더 간 겁니다.

STEP 4. 검수 — 세 개의 눈

도구가 늘어도 직접 검수는 남습니다. 세 눈으로 봅니다.

```
# ① 코드 읽는 눈 (완성도) — Claude Code에 원래 있는 기능
/code-review
```

```
# ② 코드 읽는 눈 (보안)
/security-review
```

```
# ③ 둘러보는 눈 (지난주 내가 만든 스킴)
/check  동시 접근과 실시간 채팅
```

⚠ `/code-review` · `/security-review` 는 superpowers가 아니라 **Claude Code**에 원래 들어있는 기능이에요(설치 안 해도 있음). `/check` 는 지난주 내가 만든 것.

세 눈이 찾은 걸 고치기:

방금 세 검수에서 나온 것 중 중요한 것부터 고쳐줘. 뭘 고쳤는지 한 줄씩 알려줘

💡 **핵심:** 리뷰 스킵은 코드를 읽어줄 뿐이에요. 실제로 돌려보고 내 눈으로 확인하는 건 여전히 내 몫. 그래서 `/check` 는 안 버립니다.

STEP 5. 클라이맥스 — 두 탭으로 실시간 확인

이번엔 시연이 곧 증명이에요.

1. 브라우저 탭 2개로 같은 채팅방 열기
2. 한쪽에서 메시지 입력 → 전송

✅ **확인:** 새로고침 없이 다른 쪽 화면에 메시지가 바로 뜨면 성공. (안 뜨면 → 새로고침해서 저장은 됐는지 먼저 확인 → SI에게 **"실시간 반영이 안 된다"**고 알리고 고쳐달라고.)

봉인 — 오늘 이름표 남기기:

지금 커밋에 `day3-done` 태그 붙여줘

✅ **확인:** 태그 완료 답이 오면 성공.

막혔을 때 대처법

이런 상황	이렇게 하세요
설치가 막혀요	"플러그인 마켓플레이스로 설치하는 법 찾아줘 (obra/superpowers-marketplace)" *
brainstorming이 안 끝나요	정상이에요. 열 번 넘게 답하는 게 맞아요. 모르면 AI 추천 고르기
빨간불(●)이 떠서 놀랐어요	TDD에선 정상 이에요. 채점표가 살아있다는 뜻
실시간이 안 떠요	새로고침해서 저장은 됐는지 확인 → "실시간 반영이 안 된다, 고쳐줘" *
뭔가 망가졌어요	방금 커밋으로 되돌려줘 / <code>day3-done</code> 태그로 되돌려줘
영어 에러가 났어요	그 에러를 복사해서 AI에게 붙여넣고 "이 에러 고쳐줘" *

혼자 더 해보기

- [] superpowers를 4점검 후 설치해보기 (점검 프롬프트부터)
 - [] brainstorming으로 내 사업의 기능 하나 구상해보기 (미니당근 말고 내 것)
 - [] 발사할 때 꼭 "TDD로 진행해줘" 붙이기 → ● → ● 왕복 눈으로 보기
 - [] 세 눈(/code-review · /security-review · /check)으로 검수하고 고치기
-

📖 용어 미니사전

- **superpowers** : 프로들이 만든 스킬 상자(여러 스킬이 한 번에). 설치 전 4점검 필수.
 - **brainstorming** : AI가 하나씩 캐물어 설계도·계획서를 만들어주는 스킬.
 - **TDD** : Test-Driven Development. 만들기 전에 "이렇게 되면 합격"(채점표)을 먼저 정하는 방식.
 - **채점표(테스트)** : 결과가 맞는지 확인해주는 코드. 기능은 그 결과를 실제로 만드는 코드.
 - ● RED / ● GREEN : 빨강=채점표가 작동한다(불합격 잘 잡음) / 초록=기능이 됐다. 빨강→초록 왕복이 정상.
 - **검수 세 눈** : /code-review (완성도) · /security-review (보안) = 읽는 눈(내장) + /check (돌려보는 눈, 내 것).
 - **실시간성** : 남이 바꾼 걸 새로고침 없이 바로 보는 것 (≠ 동시성=충돌 안 나게).
 - **보안 규칙** : "누가 뭘 할 수 있나"를 서버가 강제하는 것. 화면에서 버튼 숨기는 것과 다름.
-

*오즈코딩스쿨 7기 열정반 특강 · 3회차 복습 카드 · 도구는 새것, 순서는 지난주 그대로 😊 *